TP n°5

Transactions et contrôle de concurrence

Bouattay Ferdaous

Matière : Base de données avancés

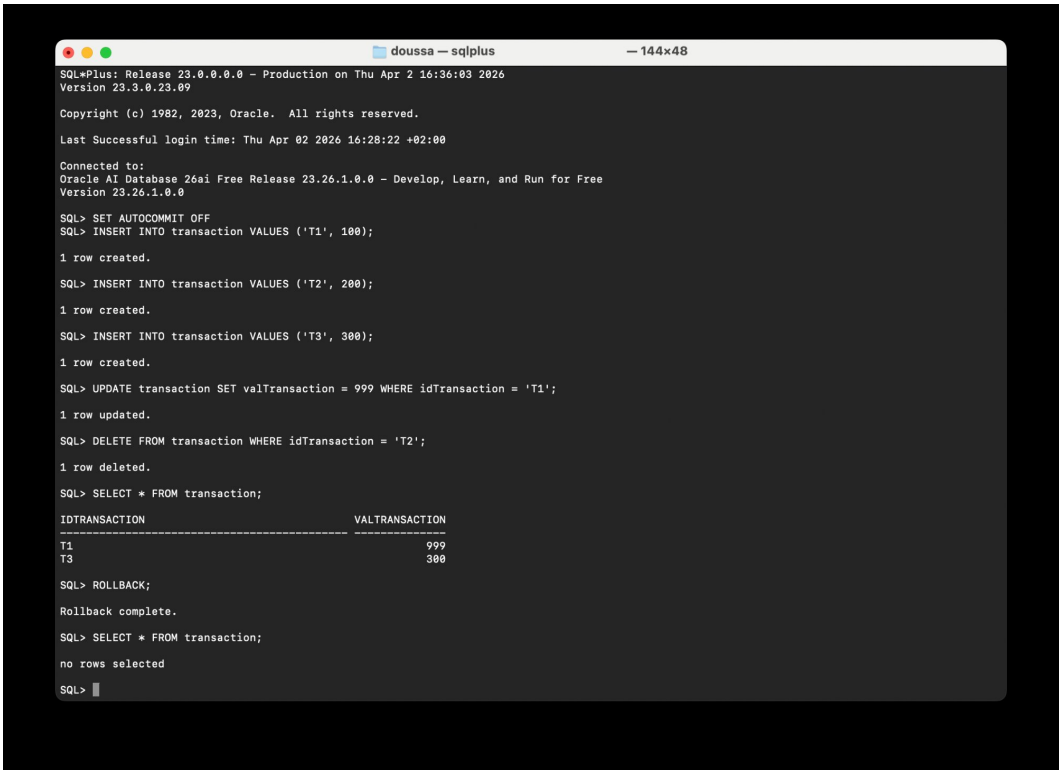
Exercice 1 – Atomicite d'une transaction

On ouvre deux sessions S1 et S2 et on execute `SET AUTOCOMMIT OFF` dans chacune avant de commencer.

Question 1 – Création de la table `transaction` dans S1

```
CREATE TABLE transaction (
    idTransaction  VARCHAR2(44),
    valTransaction NUMBER(10)
);
```

Question 2 – Insertions, modification, suppression et ROLLBACK dans S2



```
doussa — sqlplus — 144×48
SQL*Plus: Release 23.0.0.0.0 - Production on Thu Apr 2 16:36:03 2026
Version 23.3.0.23.09

Copyright (c) 1982, 2023, Oracle.  All rights reserved.

Last Successful login time: Thu Apr 02 2026 16:28:22 +02:00

Connected to:
Oracle AI Database 26ai Free Release 23.26.1.0.0 - Develop, Learn, and Run for Free
Version 23.26.1.0.0

SQL> SET AUTOCOMMIT OFF
SQL> INSERT INTO transaction VALUES ('T1', 100);

1 row created.

SQL> INSERT INTO transaction VALUES ('T2', 200);

1 row created.

SQL> INSERT INTO transaction VALUES ('T3', 300);

1 row created.

SQL> UPDATE transaction SET valTransaction = 999 WHERE idTransaction = 'T1';

1 row updated.

SQL> DELETE FROM transaction WHERE idTransaction = 'T2';

1 row deleted.

SQL> SELECT * FROM transaction;

IDTRANSACTION                                VALTRANSACTION
-------------------------------------------- --------------
T1                                                      999
T3                                                      300

SQL> ROLLBACK;

Rollback complete.

SQL> SELECT * FROM transaction;

no rows selected

SQL>
```

Apres le `ROLLBACK`, la table est vide : toutes les opérations ont été annulées d'un coup. Le `ROLLBACK` restaure l'état initial de la base.

Question 3 – Insertions dans S2 puis `quit;` sans COMMIT, que voit S1 ?

```
SQL> INSERT INTO transaction VALUES ('T1', 100);

1 row created.

SQL> INSERT INTO transaction VALUES ('T2', 200);

1 row created.

SQL> quit;
Disconnected from Oracle AI Database 26ai Free Release 23.26.1.0.0 - Develop, Learn, and Run for Free
Version 23.26.1.0.0
```

S1 ne voit aucune ligne. En quittant avec `quit;` sans `COMMIT`, Oracle effectue un `ROLLBACK` automatique. Les données insérées par S2 sont perdues car elles n'ont jamais été validées.

Question 4 – Fermeture brutale de S1, données préservées ?

```
Last login: Mon Apr  6 12:25:13 on ttys002
[doussa@Btys-MacBook ~ % sqlplus system/doussa1234@localhost:1521/FREEPDB1       ]

SQL*Plus: Release 23.0.0.0.0 - Production on Mon Apr 6 12:39:50 2026
Version 23.3.0.23.09

Copyright (c) 1982, 2023, Oracle.  All rights reserved.

Last Successful login time: Mon Apr 06 2026 12:35:52 +02:00

Connected to:
Oracle AI Database 26ai Free Release 23.26.1.0.0 - Develop, Learn, and Run for F
ree
Version 23.26.1.0.0

SQL> SELECT * FROM transaction;

no rows selected
```

La table est vide après reconnexion. Une fermeture brutale déclenche un `ROLLBACK` implicite cote serveur : toute transaction non validée est annulée automatiquement.

Question 5 – Insertion + ALTER TABLE + ROLLBACK

```
SQL> INSERT INTO transaction VALUES ('B1', 202);

1 row created.

SQL> INSERT INTO transaction VALUES ('B2', 303);

1 row created.

SQL> ALTER TABLE transaction ADD val2transaction NUMBER(10);

Table altered.

SQL> ROLLBACK;

Rollback complete.

SQL> SELECT * FROM transaction;

IDTRANSACTION                                VALTRANSACTION VAL2TRANSACTION
-------------------------------------------- -------------- ---------------
B1                                                      202
B2                                                      303

SQL> DESCRIBE transaction;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 IDTRANSACTION                                      VARCHAR2(44)
 VALTRANSACTION                                     NUMBER(10)
 VAL2TRANSACTION                                    NUMBER(10)
```

Les lignes B1 et B2 sont toujours présentes et la colonne `val2transaction` a été ajoutée malgrè le `ROLLBACK`. En Oracle, tout ordre DDL (`ALTER TABLE`, `CREATE`, `DROP`) provoque un **COMMIT implicite** avant son exécution. Au moment du `ROLLBACK`, une nouvelle transaction venait de commencer : il n'y avait donc rien à annuler.

Question 6 – Conclusion

Une **session** est une connexion active entre un utilisateur et le serveur Oracle. Elle peut contenir plusieurs transactions exécutées en serie. Une **transaction** est une séquence d'opérations traitée comme une unité atomique, qui se termine par `COMMIT` (validation définitive) ou `ROLLBACK` (annulation complète). Les propriétes ACID garantissent l'atomicite, la cohérence, l'isolation et la durabilite de chaque transaction.

Exercice 2 — Transactions concurrentes

On travaille avec deux sessions ouvertes en parallèle. Le niveau d'isolation utilisé est le niveau par défaut d'Oracle : **READ COMMITTED**.

2.1 — Création du schéma

On crée les tables `vol` et `client`, puis on insère les données initiales.

```
SQL> CREATE TABLE vol (
  2    idVol VARCHAR2(44),
  3    capaciteVol NUMBER(10),
  4    nbrPlacesReserveesVol NUMBER(10)
  5  );

Table created.

SQL> CREATE TABLE client (
  2    idClient VARCHAR2(44),
  3    prenomClient VARCHAR2(11),
  4    nbrPlacesReservesClient NUMBER(10)
  5  );

Table created.
```

```
SQL> INSERT INTO vol VALUES ('V1', 100, 0);

1 row created.

SQL> INSERT INTO client VALUES ('C1', 'Alice', 0);

1 row created.

SQL> INSERT INTO client VALUES ('C2', 'Bob', 0);

1 row created.

SQL> COMMIT;

Commit complete.
```

Les deux tables sont créées. Le vol V1 a une capacité de 100 places, avec 0 réservation. Alice (C1) et Bob (C2) démarrent également à 0. Le `COMMIT valide ces données initiales.`

2.2 — Isolation des transactions

On effectue une réservation de 2 billets pour Alice dans T1, sans valider encore.

Session T1 — UPDATE pour C1, sans COMMIT :

```
SQL> UPDATE vol SET nbrPlacesReserveesVol = nbrPlacesReserveesVol + 2 WHERE idVol = 'V1';

1 row updated.

SQL> UPDATE client SET nbrPlacesReservesClient = nbrPlacesReservesClient + 2 WHERE idClient = 'C1';

1 row updated.

SQL> SELECT * FROM vol;

IDVOL                                      CAPACITEVOL NBRPLACESRESERVEESVOL
------------------------------------------ ----------- ---------------------
V1                                                 100                     2

SQL> SELECT * FROM client;

IDCLIENT                                   PRENOMCLIEN NBRPLACESRESERVESCLIENT
------------------------------------------ ----------- -----------------------
C1                                         Alice                             2
C2                                         Bob                               0
```

Session T2 — lecture pendant que T1 n'a pas encore commité :

```
Connected to:
Oracle AI Database 26ai Free Release 23.26.1.0.0 - Develop, Learn, and Run for Free
Version 23.26.1.0.0

SQL> SELECT * FROM vol;

IDVOL                                      CAPACITEVOL NBRPLACESRESERVEESVOL
------------------------------------------ ----------- ---------------------
V1                                                 100                     0

SQL> SELECT * FROM client;

IDCLIENT                                   PRENOMCLIEN NBRPLACESRESERVESCLIENT
------------------------------------------ ----------- -----------------------
C1                                         Alice                             0
C2                                         Bob                               0
```

T1 voit ses propres modifications (nbrPlaces = 2). T2 voit toujours 0 : les modifications non validées de T1 lui sont invisibles. C'est le principe d'**isolation — chaque transaction travaille sur sa propre vue de la base.**

2.3 — COMMIT et ROLLBACK

T1 fait ROLLBACK — vérification dans T2 :

```
SQL> UPDATE vol SET nbrPlacesReserveesVol = nbrPlacesReserveesVol + 2 WHERE idVol = 'V1';

1 row updated.

SQL> UPDATE client SET nbrPlacesReservesClient = nbrPlacesReservesClient + 2 WHERE idClient = 'C1';

1 row updated.

SQL> COMMIT;

Commit complete.
```

Après le `ROLLBACK de T1, T2 voit toujours 0. La base est revenue à son état initial, comme si T1 n'avait jamais existé.`

T1 refait la réservation et la valide avec COMMIT :

```
SQL> SELECT * FROM vol;

IDVOL                                      CAPACITEVOL NBRPLACESRESERVEESVOL
------------------------------------------ ----------- ---------------------
V1                                                 100                     2

SQL> SELECT * FROM client;

IDCLIENT                                   PRENOMCLIEN NBRPLACESRESERVESCLIENT
------------------------------------------ ----------- -----------------------
C1                                         Alice                             2
C2                                         Bob                               0
```

T2 lit la base après le COMMIT de T1 :

```
SQL> UPDATE vol SET nbrPlacesReserveesVol = 0 WHERE idVol = 'V1';

1 row updated.

SQL> UPDATE client SET nbrPlacesReservesClient = 0 WHERE idClient IN ('C1', 'C2');

2 rows updated.

SQL> COMMIT;

Commit complete.
```

Cette fois T2 voit bien nbrPlaces = 2. Les modifications de T1 sont visibles car validées. Un `ROLLBACK n'aurait plus aucun effet : c'est la durabilité. Cela confirme que le niveau par défaut d'Oracle est READ COMMITTED - chaque SELECT voit la dernière version validée.`

2.4 — Mises à jour perdues (Lost Update)

On réinitialise les données à 0, puis on exécute T1 et T2 en parallèle dans l'ordre : sélections T1 → sélections T2 → UPDATE + COMMIT T1 → UPDATE + COMMIT T2.

Réinitialisation :

```
SQL> SELECT nbrPlacesReserveesVol FROM vol WHERE idVol = 'V1';

NBRPLACESRESERVEESVOL
---------------------
                    0

SQL> SELECT nbrPlacesReservesClient FROM client WHERE idClient = 'C1';

NBRPLACESRESERVESCLIENT
-----------------------
                      0
```

Étape 1 — T1 lit les valeurs (obtient 0) :

```
SQL> SELECT nbrPlacesReserveesVol FROM vol WHERE idVol = 'V1';

NBRPLACESRESERVEESVOL
---------------------
                    0

SQL> SELECT nbrPlacesReservesClient FROM client WHERE idClient = 'C2';

NBRPLACESRESERVESCLIENT
-----------------------
                      0
```

Étape 2 — T2 lit les valeurs (obtient aussi 0) :

```
SQL> UPDATE vol SET nbrPlacesReserveesVol = 0 + 2 WHERE idVol = 'V1';

1 row updated.

SQL> UPDATE client SET nbrPlacesReservesClient = 0 + 2 WHERE idClient = 'C1';

1 row updated.

SQL> COMMIT;

Commit complete.
```

Étape 3 — T1 met à jour (+2 billets) et commite :

```
IDVOL                                    CAPACITEVOL NBRPLACESRESERVEESVOL
---------------------------------------- ----------- ---------------------
V1                                               100                     3

SQL> SELECT * FROM client;

IDCLIENT                                 PRENOMCLIEN NBRPLACESRESERVESCLIENT
---------------------------------------- ----------- -----------------------
C1                                       Alice                             2
C2                                       Bob                               3
```

Étape 4 — T2 met à jour (+3 billets) et commite, puis vérification :

```
SQL> UPDATE client SET nbrPlacesReservesClient = 0 WHERE idClient IN ('C1', 'C2');

2 rows updated.

SQL> COMMIT;

Commit complete.

SQL> SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;

Transaction set.
```

Les deux clients ont réservé 2+3 = 5 billets, mais le vol n'affiche que **3 au lieu de 5. T2 avait lu 0 avant le COMMIT de T1 et a calculé 0+3=3, ignorant la réservation d'Alice. C'est le problème des mises à jour perdues : la mise à jour de T1 a été écrasée silencieusement. La base est dans un état incohérent.**

2.5 — Mode SERIALIZABLE

On réinitialise et on rejoue la même exécution avec `SET TRANSACTION ISOLATION LEVEL SERIALIZABLE` dans les deux sessions.

```
SQL> UPDATE vol SET capaciteVol = 10 WHERE idVol = 'V1';

1 row updated.

SQL> COMMIT;

Commit complete.

SQL> SELECT * FROM vol WHERE idVol = 'V1';

IDVOL                                     CAPACITEVOL NBRPLACESRESERVEESVOL
----------------------------------------- ----------- ---------------------
V1                                                 10                     5
```

Oracle détecte le conflit et rejette T2 avec l'erreur `ORA-08177 : can't serialize access for this transaction . La cohérence est garantie au prix d'un rejet éventuel - T2 devra être relancée.`

2.6 – Sequence concurrente du cours : $r_1(d)\ w_2(d)\ w_2(d')\ C_2\ w_1(d')\ C_1$

T1 lit d, puis T2 ecrit d et d' et commite, enfin T1 ecrit d' et commite. T1 écrase ainsi l'ecriture de T2 sur d' car elle n'avait pas tenu compte des modifications de T2 lors de sa lecture initiale.

Cette séquence est **acceptée par Oracle en READ COMMITTED** : T1 n'a pose aucun verrou lors de sa lecture, T2 a donc pu écrire librement.

Conclusion – Oracle utilise-t-il le 2PL vu en cours ?

Non. Oracle utilise le **MVCC** (Multi-Version Concurrency Control) et non le vérrouillage a deux phases (2PL) strict. En 2PL, chaque lecture pose un verrou partage et chaque ecriture un verrou exclusif, conserves jusqu'au COMMIT — ce qui garantit la serialisabilite mais peut causer des deadlocks. En MVCC, les lecteurs ne bloquent pas les ecrivains : en COMMITTED on risque des mises a jour perdues (cf. 2.4) ; en SERIALIZABLE Oracle leve ORA-08177 en cas de conflit.